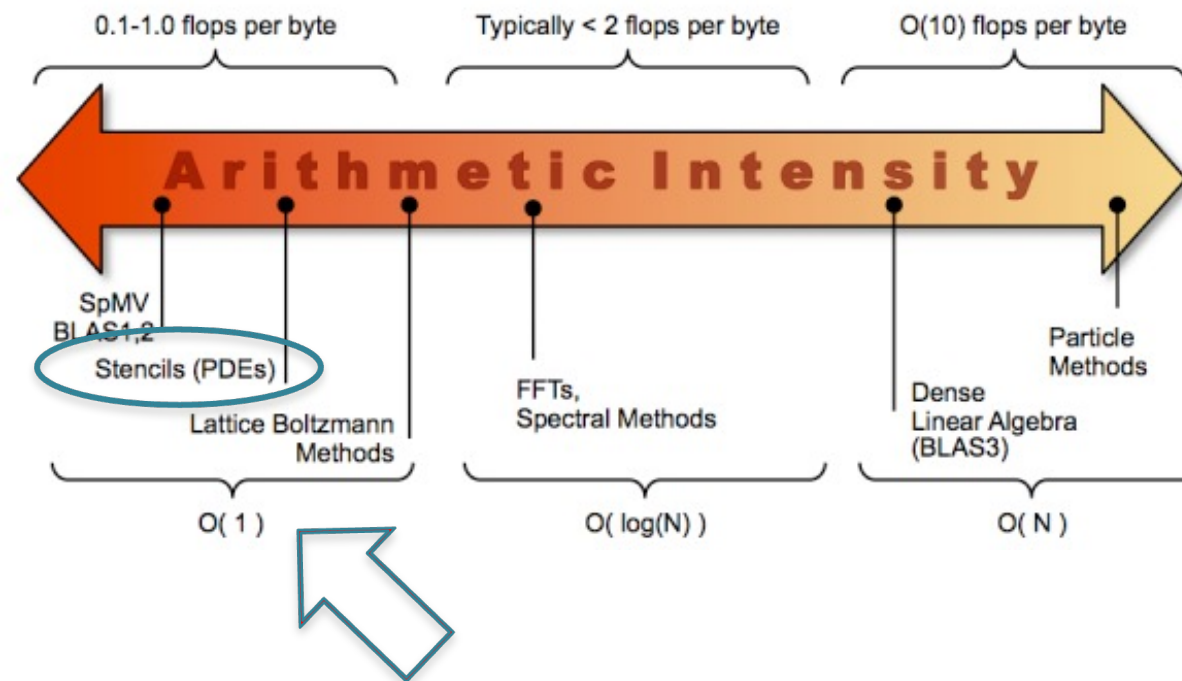


Stencil operations

Parallel Algorithms

João Luís Sobral (jls@di ...)

March/2026



Parallel Algorithms - recap

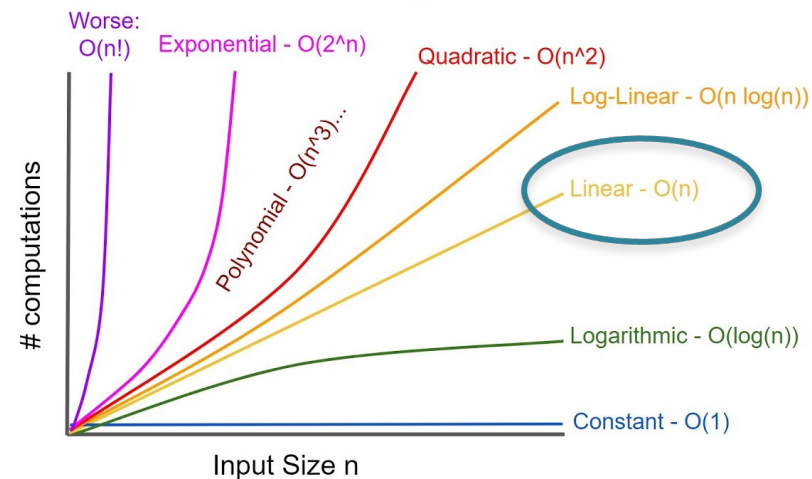
- **Traditional algorithm analysis:** Big O notation
 - Analysis of the number of operations per datum (n)
 - Particle Methods (naïve!): $O(n^2)$
 - Matrix multiplication: $\Theta(n^3)$ $O(n^{\frac{3}{2}})$ (n=number of elements of C)
 - Sorting (n keys)
 - Bad sorting (brute force): $O(n^2)$
 - Better algorithm: $O(n \log_2(n))$ & also Spectral Methods (FFT)
 - Best: $O([k]n)$

- **Stencils**

- **$O(n)$**



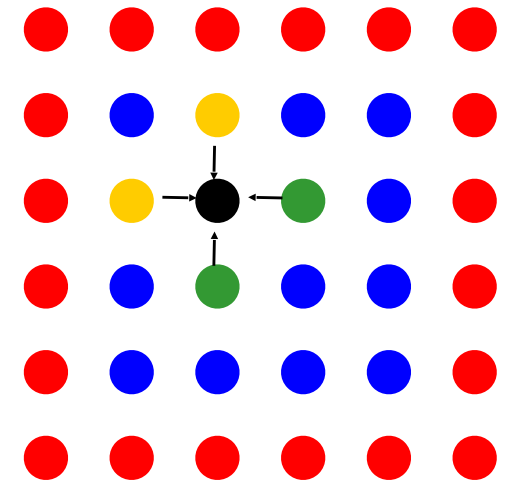
Today's lecture



Parallel Algorithms

- Typical stencil operation (2D)

```
for(i=1, i<N-1, i++)
  for(j=1, j<N-1, j++)
    A[i,j] = 0,2 x (
      A[i-1,j] +
      A[i,j-1] + A[i,j] + A[i,j+1]
      + A[i+1,j] );
```



- Instruction Level Parallelism (ILP)

- Read multiple values of A[...] from memory in parallel?
- Perform multiple arithmetic operations in parallel (which?)
- **Multiply (by 0,2?) and write A[i,j] after all operations are done**
- **How to improve ILP?**
 - Can we compute A_{i,j} and A_{i,j+1} in parallel?
- **What dependency constrains the computing of multiple elements of A in parallel?**

```
A[i,j+1] = 0,2 x (
  A[i-1,j+1] +
  A[i,j] + A[i,j+1] + A[i,j+2]
  + A[i+1,j+1] );
```

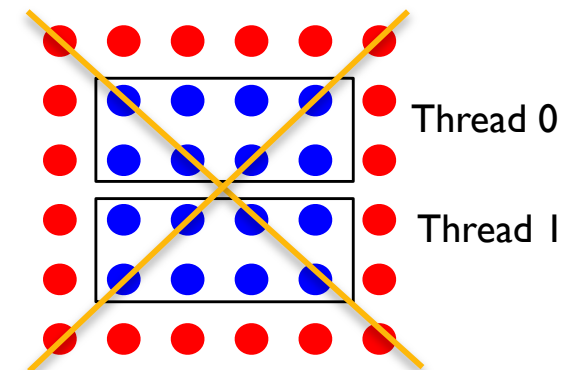
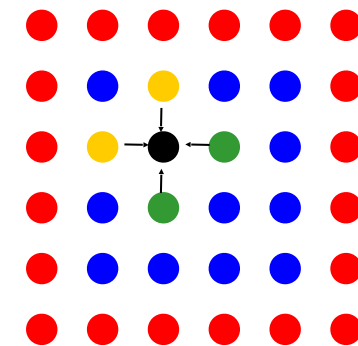
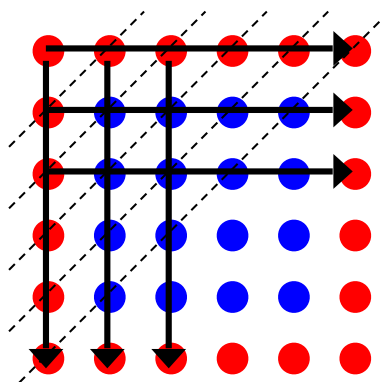
Parallel Algorithms

```
for(i=1, i<N-1, i++)
  for(j=1, j<N-1, j++)
    A[i,j] = 0,2 x (
      A[i-1,j] +
      A[i,j-1] + A[i,j] + A[i,j+1]
      + A[i+1,j] );
```

- Thread-level [task] parallelism

- Each thread [task] computes a set of rows of the matrix

- Data dependencies



Does not respect
data dependencies
(introduces a **race
condition**)

Stencil examples (WA2 - CP 2024/2025)

Optimised solver

a) red & black
phases

b) detects early
convergence

```
void lin_solveRedBlack(int M, int N, int O, int b, float *x, float *x0, float a, float c) {
    float tol = 1e-7, max_c, old_x, change;
    int l = 0;

    do {
        max_c = 0.0f;
        for (int i = 1; i <= M; i++) {
            for (int j = 1; j <= N; j++) {
                for (int k = 1 + (i+j)%2; k <= O; k+=2) {
                    old_x = x[IX(i, j, k)];
                    x[IX(i, j, k)] = (x0[IX(i, j, k)] +
                                     a * (x[IX(i - 1, j, k)] + x[IX(i + 1, j, k)] +
                                           x[IX(i, j - 1, k)] + x[IX(i, j + 1, k)] +
                                           x[IX(i, j, k - 1)] + x[IX(i, j, k + 1)])) / c;

                    change = fabs(x[IX(i, j, k)] - old_x);
                    if (change > max_c) max_c = change;
                }
            }
        }

        for (int i = 1; i <= M; i++) {
            for (int j = 1; j <= N; j++) {
                for (int k = 1 + (i+j+1)%2; k <= O; k+=2) {
                    old_x = x[IX(i, j, k)];
                    x[IX(i, j, k)] = (x0[IX(i, j, k)] +
                                     a * (x[IX(i - 1, j, k)] + x[IX(i + 1, j, k)] +
                                           x[IX(i, j - 1, k)] + x[IX(i, j + 1, k)] +
                                           x[IX(i, j, k - 1)] + x[IX(i, j, k + 1)])) / c;

                    change = fabs(x[IX(i, j, k)] - old_x);
                    if (change > max_c) max_c = change;
                }
            }
        }

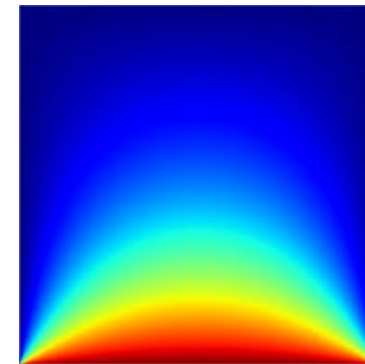
        set_bnd(M, N, O, b, x);
    } while (max_c > tol && ++l < 20);
}
```

Stencil examples

- Simulação do processo de difusão de calor, para N_MAX iterações

- Pseudo-código:

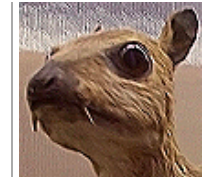
```
iteration() {  
    for(int i=1; i<N-1; i++) {  
        for(int j=1; j<N-1; j++)  
            G1[i][j]= (  
                G2[i-1][j]+  
                G2[i+1][j]+  
                G2[i][j-1]+  
                G2[i][j+1]+  
                G2[i][j]) / 5;  
        }  
    }  
    ... // copia G1 para G2
```



Stencil examples

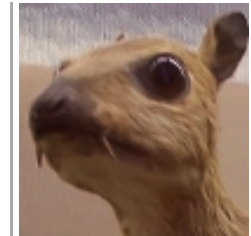
- Convolução para melhorar a qualidade de uma imagem (máscaras de 3x3, 5x5 ou 7x7)

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$



Identity

$$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$



- Algoritmo:

```
for each image row in input image:
  for each pixel in image row:

    set accumulator to zero

    for each kernel row in kernel:
      for each element in kernel row:

        if element position corresponding* to pixel position then
          multiply element value corresponding* to pixel value
          add result to accumulator
        endif

    set output image pixel to accumulator
```

Stencil examples

- Esqueletização de uma imagem binária

Morphological Image processing

- Algoritmo:

1. Numerar vizinhança de P_1

	P_9	P_2	P_3
	P_8	P_1	P_4
	P_7	P_6	P_5

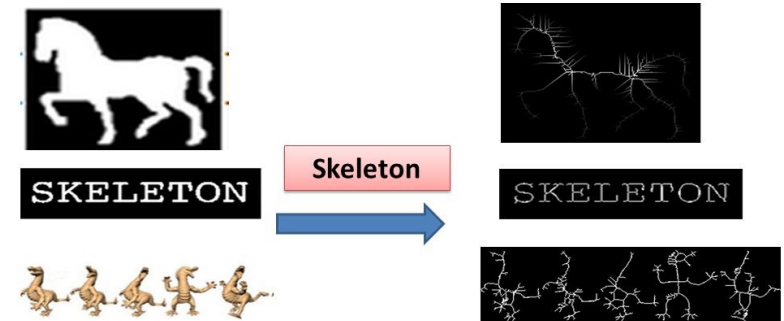
2. 1º passo – remover se:

- o número de vizinhos brancos de P_1 (P_2, P_3, P_4 , etc) estiver no intervalo [2..6]
- só existir uma transição branco-preto na vizinhança de P_1 (transição P_2 - P_3 , P_3 - P_4 , P_4 - P_5 , etc);
- P_4 =preto ou P_6 =preto ou P_2 e P_8 são ambos pretos

3. 2º passo - repetir 1º passo alterando condição 3.) para

- se P_2 =preto ou P_8 =preto ou P_4 e P_6 são ambos pretos

4. Repetir 2. e 3. até não serem removidos mais pixels



Performance of parallel applications

- **Performance models**

- Make it possible to compare the complexity of algorithms, their scalability and identify bottlenecks before a considerable time is invested in implementation

- What is the definition of performance?

- There are multiple alternatives:
 - Execution time, efficiency, scalability, memory requirements, throughput, latency, project costs / development costs, portability, reuse potential
 - The importance of each one depends on the concrete application
- Most common measure in parallel applications is speed-up: t_{seq}/t_{par}

- Amdahl law

- The sequential component of an application limits the maximum speed-up
 - If s is the sequential fraction of an algorithm then the maximum possible gain is $1/s$.
- Reinforces the idea that we should prefer algorithms suitable for parallel execution: think parallel.

Performance of parallel applications

- **Performance models**

- Should explain observations and predict behaviour
 - Defined as a function of the problem dimension, number of processing units (PU), number of tasks, etc.
- Execution time
 - Time measured since the first process (or thread) starts execution until the last process (thread) terminates (wall time)
 - $T_{exec} = T_{comp} + T_{comm} + T_{free}$
 - **Computation time** – time spent in computations
 - Excludes communication/synchronization and free time.
 - The sequential version can be used to estimate T_{comp} .
 - **Free time** - when a PU becomes starved (without work)
 - Can be complex to measure since it depends on the order of tasks
 - Can be minimized with adequate load distribution and/or overlapping computation and communication

Performance of parallel applications

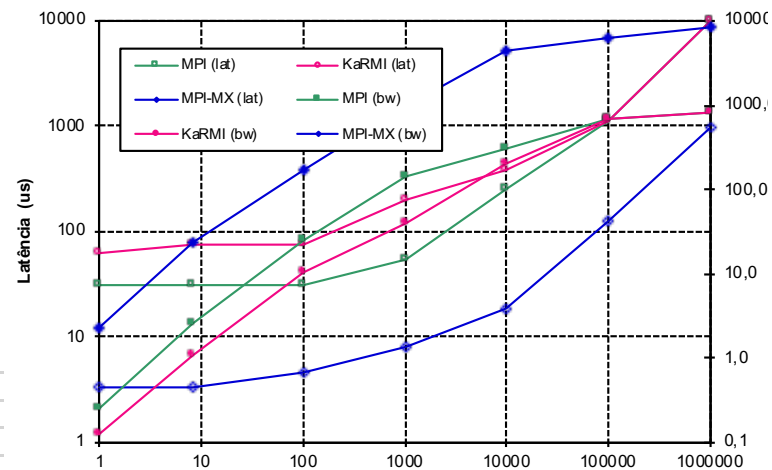
• Performance models (cont.)

- *Communication time* – time that PUs spend in sending/receiving data
 - Computed using communication latency (t_s) and throughput ($1/tw$):

- **$T_{mens} = t_s + twL$**

t_s and tw can be obtained experimentally, by a ping-pong test and a linear regression.

Tamaho	MPI (lat)	MPI (bw)	MPI-MX (lat)	MPI-MX (bw)	KaRMI (lat)	KaRMI (bw)
1	31	0,3	3,4	2,3	63	0,1
8	31	2,6	3,4	23,7	75	1,1
100	31	25,6	4,7	168,7	75	10,7
1000	55	146,3	8,1	983,9	200	40,0
10000	258	310,3	18,4	4355,9	387	206,0
100000	1136	703,8	125,5	6373,0	1137	703,0
1000000	9859	811,4	970,2	8246,0	9787	817,0
t_s (us)	31		3		63	
tw (us)	0,010		0,001		0,010	



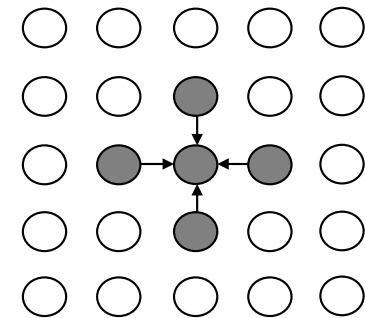
Performance of parallel applications

- **Performance models – Example Jacobi Method**

- Iterative method, at each iteration the new matrix value (X^{t+1}) is computed as the average of neighbour values from previous iteration (X^t)

$$X_{i,j}^{(t+1)} = aX_{i,j}^{(t)} + b(X_{i-1,j}^{(t)} + X_{i,j-1}^{(t)} + X_{i+1,j}^{(t)} + X_{i,j+1}^{(t)})$$

```
for(int t=0; t<Niter; t++) {  
    for(int i=1; i<N-1; i++)  
        for(int j=1; j<N-1; j++)  
            r[i][j] = a*x[i][j]+ b*(x[i-1][j] + x[i+1][j] + x[i][j-1] + x[i][j+1]);  
    // x = r on the next iteration  
}
```



Performance of parallel applications

- Performance models – **Example Jacobi Method**
 - Execution time of one iteration for a $N \times N$ matrix, on P processors, using a partition by rows with N/P rows per processor.

$$\begin{aligned}\mathbf{T_{comp}} &= \text{operations per element} \times n^\circ \text{ elements per processor} \times t_c \\ &= 6 \times (N \times N/P) \times t_c \quad (t_c = \text{time for an arithmetic operation}) \\ &= 6 t_c N^2 / P\end{aligned}$$

$$\begin{aligned}\mathbf{T_{comm}} &= \text{messages per processor} \times \text{time required for each message} \\ &= 2 \times (t_s + t_w N)\end{aligned}$$

$$\mathbf{T_{free}} = 0, \text{ since in this problem the workload is well distributed}$$

$$\begin{aligned}\mathbf{T_{exec}} &= \mathbf{T_{comp}} + \mathbf{T_{comm}} + \mathbf{T_{free}} \\ &= 6t_c N^2/P + 2t_s + 2t_w N \\ &= O(N^2/P + N)\end{aligned}$$

Performance of parallel applications

- Performance models – Example (cont)
 - **Speed-up (G)** indicates the reduction in execution time attained in P processors
 - Ratio between the best sequential algorithm and the execution time of the parallel version

$$\text{speed-up} = T_{\text{seq}} / T_{\text{par}}$$

- **Efficiency (E)** gives the fraction of time that processors perform useful work:

$$E = T_{\text{seq}} / (P \times T_{\text{par}})$$

- Jacobi case:

$$\text{Gain} = \frac{6t_c N^2}{6t_c N^2 / P + 2t_s + 2t_w N}$$

$$E = \frac{6t_c N^2}{6t_c N^2 + 2Pt_s + 2Pt_w N}$$

Performance of parallel applications

- Scalability analysis

- Execution time, speed-up and efficiency can be used for quantitative analysis of performance

- In Jacobi example:

- Execution time decreases when P increases, but it is limited by the time to exchange two lines

$$T_{\text{exe}} = 6t_c N^2 / P + 2t_s + 2t_w N$$

- Execution time increases with N, t_c , t_s and t_w

$$G = \frac{6t_c N^2}{6t_c N^2 / P + 2t_s + 2t_w N}$$

- Efficiency decreases when P, t_s e t_w increase

- Efficiency increases with N and t_c ;

$$E = \frac{6t_c N^2}{6t_c N^2 + 2Pt_s + 2Pt_w N}$$

- Scalability for problems with fixed size

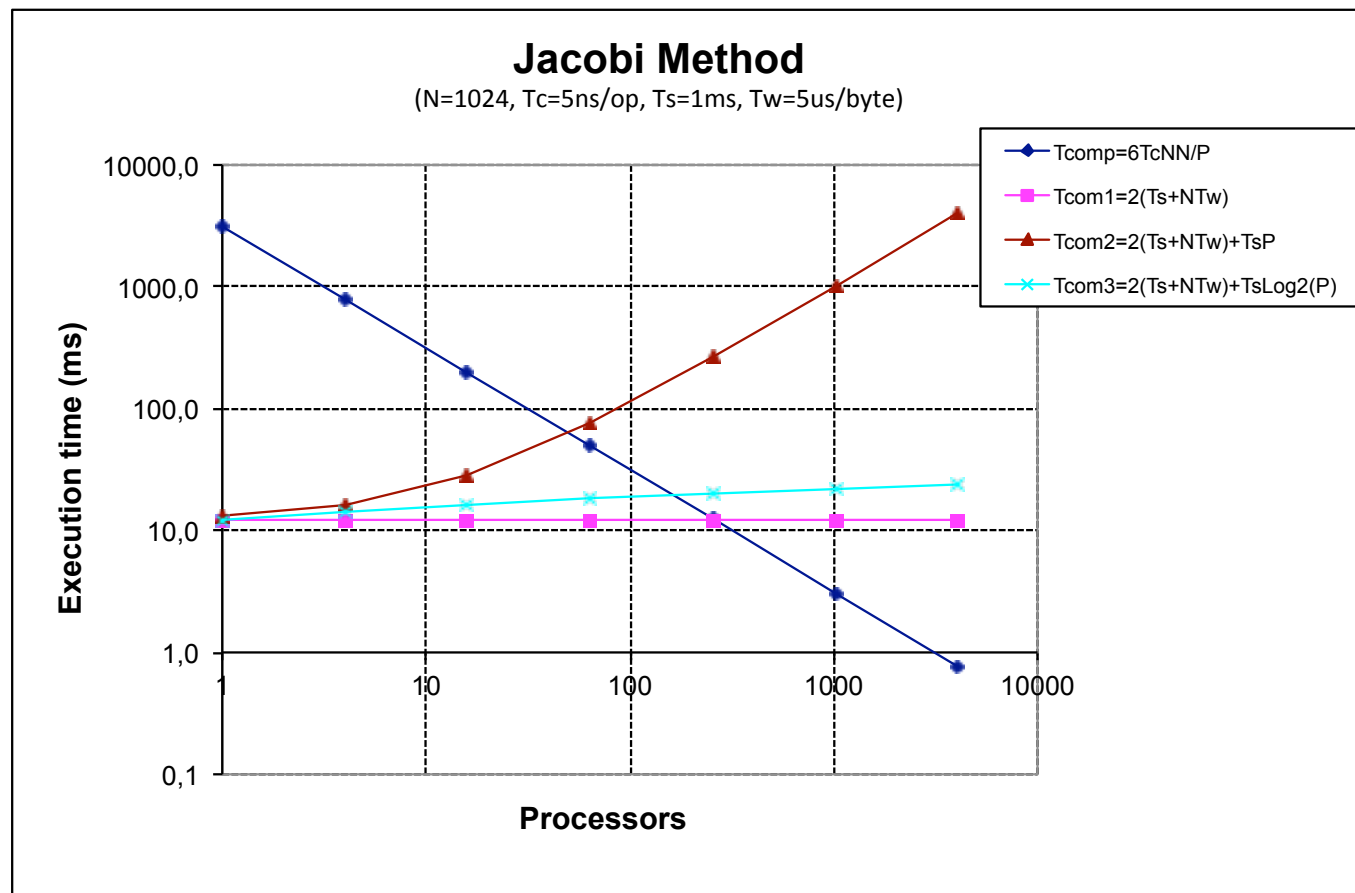
- Analysis of T_{exe} and E when P increases
- In general, E decreases, **T_{exe} can increase if it has a positive power of P**

- Scalability for problems with variable size (weak scalability)

- In some cases, more processors are used to solve larger problems, keeping the same efficiency levels
- Isoefficiency indicates what is the required increase in the problem dimension, to keep the same efficiency, when the number of processors increases

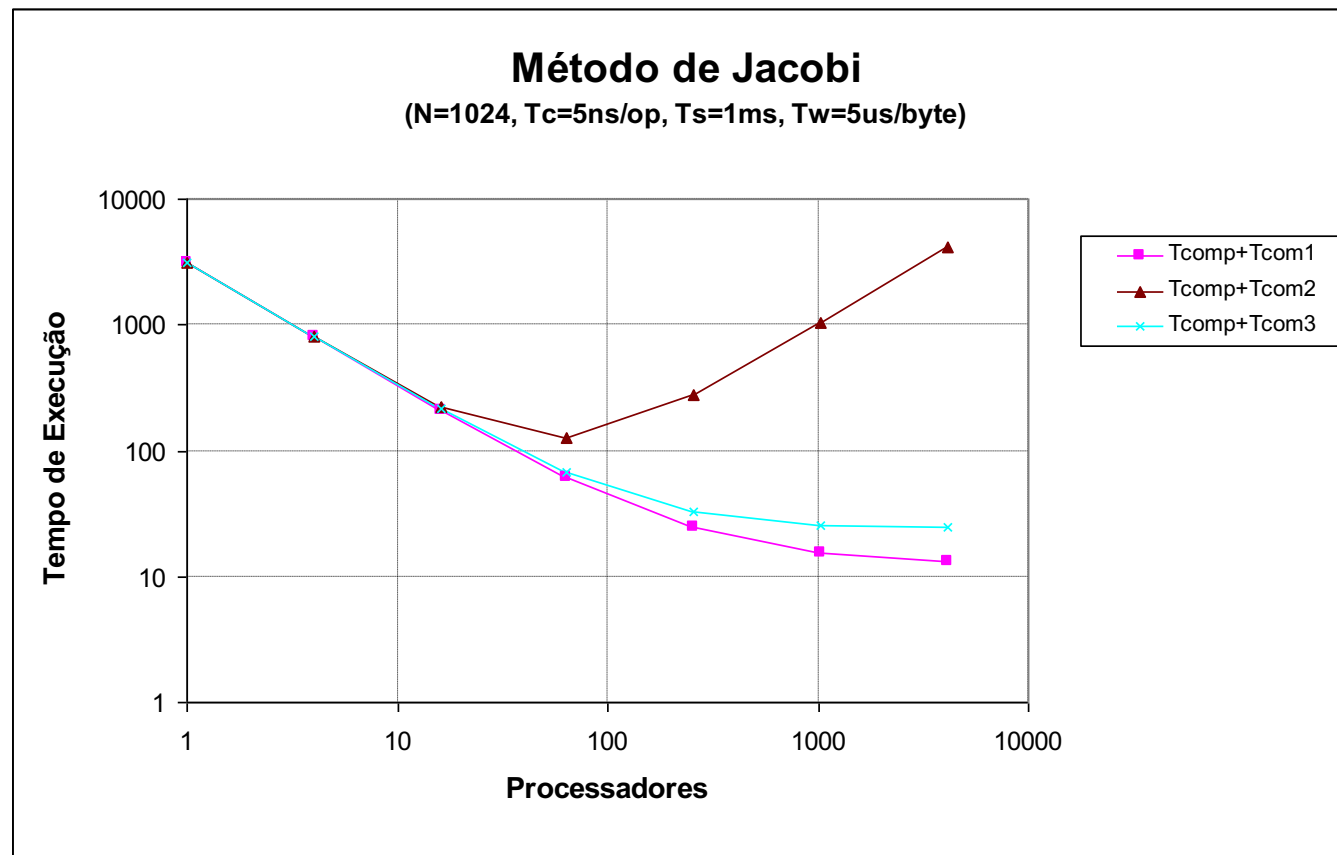
Performance of parallel applications

- Scalability analysis (cont)



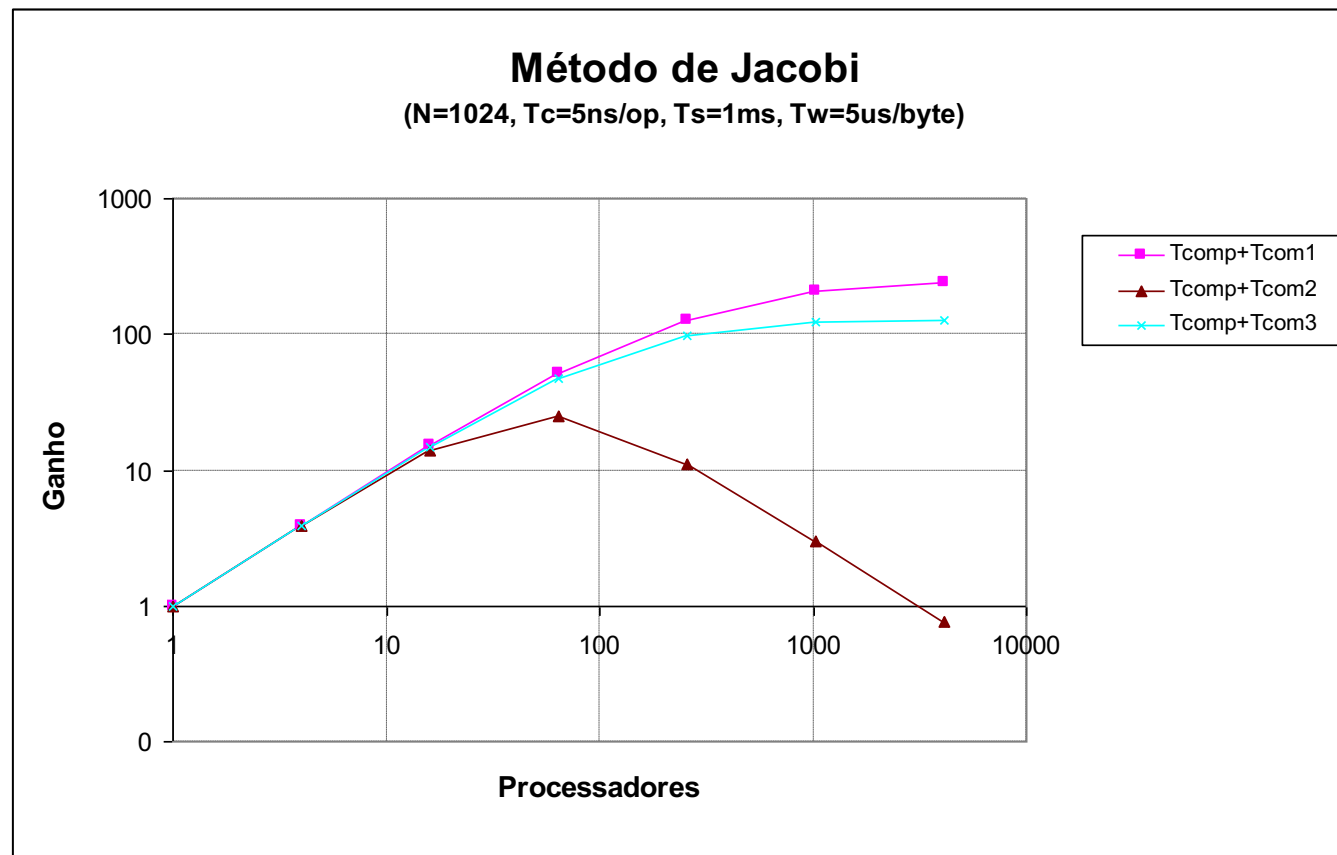
Performance of parallel applications

- **Scalability analysis (cont)**



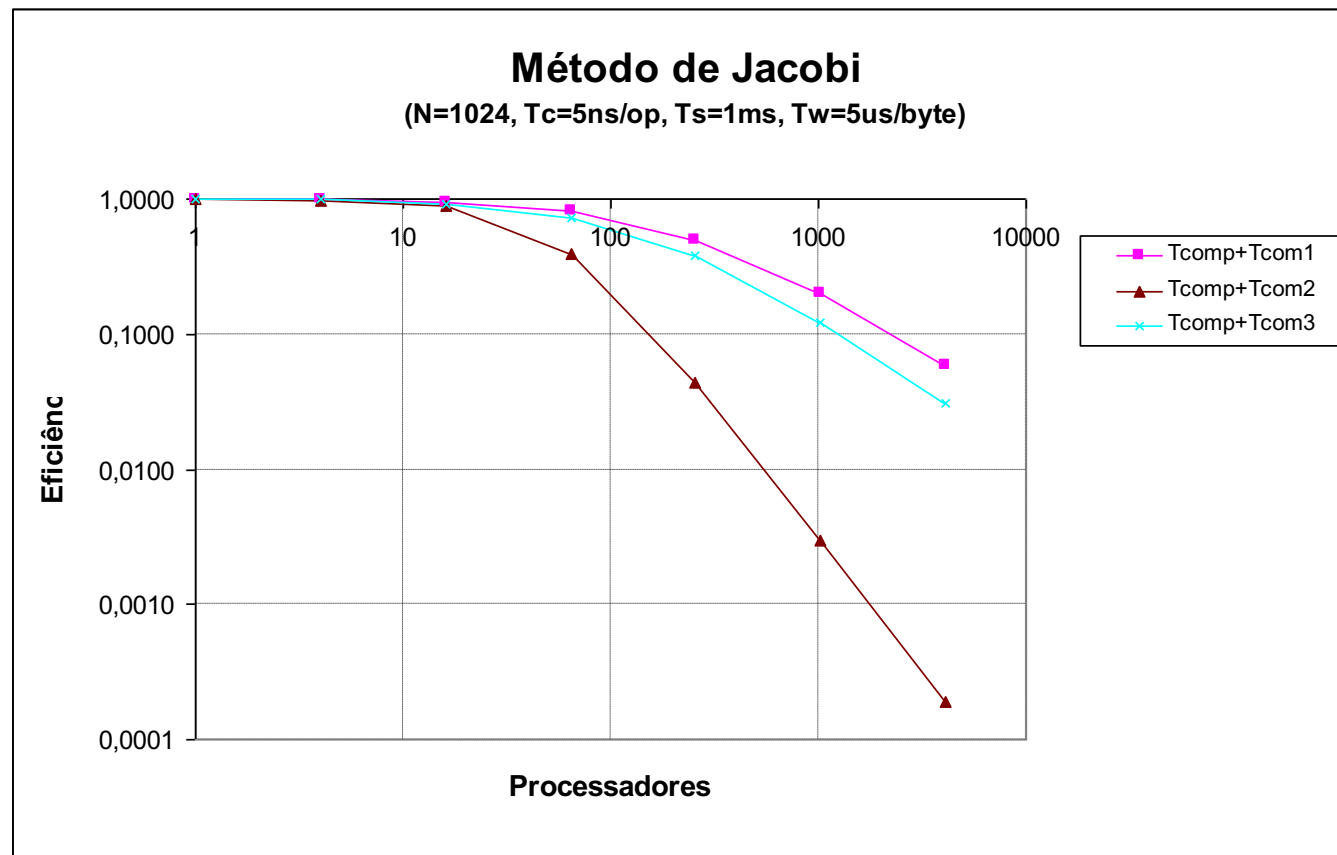
Performance of parallel applications

- **Scalability analysis (cont)**



Performance of parallel applications

- **Scalability analysis (cont)**



PL lecture & project WA2

Parallelize the given **Jacobi Method C code**

- Shared Memory: OpenMP & gcc
- Measure speed-up for input of size of 1024 & 4096

Project (draft)

1. Compare the base (sequential) performance of Jacobi, SOR and SOR-RB methods
 - 1st phase: same number of computations in all methods
2. Perform a scalability analysis with OpenMP
3. Develop a MPI version

Evaluation

50% [teorical] problem analysis (MPI-based)
30% experimental setup & performance tests
20% report

Deadline: 29-Março